Notebook 07: **Video & GIF Asset Builder**

Purpose of This Notebook

This notebook marks a transition in the final pipeline:

> from **analysis** → to **artifact generation**

All previous notebooks (01–06) established:

- a **validated baseline** (Notebook 01),
- a **multi-window temporal structure** (Notebook 02),
- a **set of proxy formulations and their behavior** (Notebook 03),
- **global signal consistency across windows** (Notebook 04),
- **semantic structure and separability analysis** (Notebook 05),
- and explicit **failure-case understanding** (Notebook 06).

This notebook does **not** introduce new analysis, metrics, or claims.

Instead, it answers:

> How do we **visually communicate** the full story of the project in a clean, honest, and interpretable way?

Role in the Final Pipeline

This notebook implements the visualization layer described in the project planning:

- transforms validated signals into **frame-wise visual artifacts**.
- constructs **consistent rendering primitives**.
- produces **GIF-ready and video-ready outputs**.

It directly supports:

- final report figures.
- qualitative evaluation.
- interpretability of proxy behavior.

This is not debugging visualization.
This is **publication-grade artifact construction**.

Core Visualization Concepts

We operate on the same signals established earlier:

- Raw intensity: I

- Range: $R = \mathrm{sqrt}(x^2 + y^2 + z^2)$
- Pseudo-reflectivity proxy: $\hat{\rho} = I \cdot R$

Important constraint:

> $\hat{\rho}$ is **not calibrated reflectivity**,
> but a **heuristic range-aware transformation** used for interpretability.

This assumption remains unchanged.

Visualization Objectives

We construct two primary visualization boards:

1. Signal Comparison Board

A structured comparison across:

BEV (Top-down)

- geometry (structure only).
- raw intensity.
- pseudo-reflectivity.

Forward View

- geometry.
- raw intensity.
- pseudo-reflectivity.

Goal:

- visually inspect how the proxy reshapes signal structure.
- compare spatial emphasis and range behavior.
- maintain consistency across frames.

2. Semantic Reflectivity Overlay

A combined visualization where:

- **color → semantic class**.
- **brightness → proxy signal ($\hat{\rho}$)**.

Includes:

- forward view.
- BEV view.
- legend panel.
- class distribution summary.

Goal:

- observe how proxy interacts with semantic structure.
- reveal class-wise signal variation.
- maintain interpretability without distorting class identity.

Rendering Principles (Strict)

All visualizations must obey:

1. **Fixed spatial limits**
   - no per-frame zoom.
   - no dynamic cropping.
2. **Fixed signal scaling**
   - no per-frame normalization.
   - only global or percentile-based normalization allowed.
3. **Consistent colormaps**
   - intensity vs proxy must remain comparable.
4. **Clarity over density**
   - subsample if needed.
   - avoid visual clutter.
5. **Faithful representation**
   - do not smooth away failure cases.
   - do not exaggerate proxy effects.

Mathematical Consistency

For each frame or window:

- Coordinates: $p_i = (x_i, y_i, z_i)$
- Range: $R_i = sqrt(x_i^2 + y_i^2 + z_i^2)$
- Signals: I_i (raw intensity)
  $\hat{\rho}_i = I_i \cdot R_i$

Rendering functions operate as:

- BEV projection: $(x, y) \rightarrow$ image plane
- Forward projection: (x, z) or $(x, elevation) \rightarrow$ image plane
- Signal mapping: value → colormap → pixel intensity
- Semantic overlay: color(class) × brightness($\hat{\rho}$ normalized)

Expected Outputs

By the end of this notebook, we should have:

Rendering Primitives

- reusable BEV renderer.
- reusable forward-view renderer.
- consistent normalization pipeline.

Visualization Boards

- signal comparison frames.
- semantic reflectivity overlay frames.

Motion Artifacts

- frame sequences across windows.
- GIF-ready frame buffers.

Exportable Outputs

- clean figure boards.
- animation-ready sequences.

What This Notebook Does NOT Do

This notebook does not:

- compute new metrics.
- evaluate proxy performance.
- modify semantic analysis.
- introduce new proxy variants.
- claim improvements.

All conclusions remain grounded in:

> Notebook 01–06 outputs

Final Objective

This notebook should produce:

> clean, consistent, and honest visual artifacts.
> that **faithfully represent** the behavior of reflectivity-augmented LiDAR signals.

No exaggeration.
No hiding failure.
No aesthetic over truth.

Just clear visual evidence.

Define Global Signal Normalization

Before rendering anything, we must fix one of the most critical issues:

inconsistent signal scaling across frames.

If we allow per-frame normalization:

- intensity and proxy will look artificially stable.
- temporal comparisons become meaningless.
- proxy behavior gets visually distorted.

So we enforce:

Global / Percentile-Based Normalization

We compute:

- global percentiles for:
  - raw intensity (I).
  - pseudo-reflectivity ($\hat{\rho} = I \cdot R$).

These will be used for:

- consistent colormap mapping.
- fair comparison across frames.
- stable video rendering.

Design Choice

We use:

- lower bound → p1.
- upper bound → p99.

This:

- removes extreme outliers.
- preserves meaningful structure.
- avoids heavy-tail distortion (important for I·R).

Output of This Step

We will compute and store:

- INTENSITY_MIN, INTENSITY_MAX.

- PROXY_MIN, PROXY_MAX.

These will be treated as **fixed constants** for all future rendering.

No dynamic scaling after this.

```python
import numpy as np
import pandas as pd
from pathlib import Path

# Load one representative window

candidate_roots = [
    Path("../data/semantickitti_subset/dataset/sequences/00"),
    Path("../data/semantickitti/dataset/sequences/00"),
]
DATASET_ROOT = next((p for p in candidate_roots if p.exists()),
candidate_roots[0])

velodyne_dir = DATASET_ROOT / "velodyne"

# Load window metadata (from Notebook 02)
metadata_path =
Path("../results/window_metadata/sequence00_windows.csv")
window_df = pd.read_csv(metadata_path)

# Select one medium window (stable choice from earlier notebooks)
selected_row = window_df[window_df["scale"] == "medium"].iloc[0]

start_idx = int(selected_row["start_idx"])
end_idx = int(selected_row["end_idx"])

# Frame list
frame_ids = sorted([f.stem for f in velodyne_dir.glob("*.bin")])
selected_frame_ids = frame_ids[start_idx:end_idx + 1]

# Load signals

all_intensity = []
all_ranges = []

for fid in selected_frame_ids:
    bin_path = velodyne_dir / f"{fid}.bin"
    scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)

    xyz = scan[:, :3]
    intensity = scan[:, 3]
    ranges = np.linalg.norm(xyz, axis=1)

    all_intensity.append(intensity)
    all_ranges.append(ranges)

# Concatenate
I = np.concatenate(all_intensity)
R = np.concatenate(all_ranges)

# Proxy
```

```
PR = I * R

# Compute percentile bounds

def compute_bounds(x, name):
    p1 = np.percentile(x, 1)
    p99 = np.percentile(x, 99)
    print(f"{name}: p1={p1:.4f}, p99={p99:.4f}")
    return p1, p99

INTENSITY_MIN, INTENSITY_MAX = compute_bounds(I, "Intensity")
PROXY_MIN, PROXY_MAX = compute_bounds(PR, "Pseudo-Reflectivity (I*R)")

# Store for later use
NORMALIZATION_STATS = {
    "intensity": (INTENSITY_MIN, INTENSITY_MAX),
    "proxy": (PROXY_MIN, PROXY_MAX)
}

print("\nNormalization stats locked.")
```

```
Intensity: p1=0.0000, p99=0.7100
Pseudo-Reflectivity (I*R): p1=0.0000, p99=12.0388

Normalization stats locked.
```

Define Stable Signal Normalization Function

Now that global bounds are fixed, we define a reusable normalization function.

Purpose

Convert raw signals into [0, 1] range for visualization:

- intensity → normalized intensity.
- proxy → normalized proxy.

Constraints

The function must:

1. Use fixed global bounds.
2. Clip values outside [p1, p99].
3. Avoid division issues.
4. Preserve relative structure.

Mathematical Form

For a signal x:

$$x_{norm} = clip\left(\left(x - x_{min}\right) / \left(x_{max} - x_{min}\right), 0, 1\right)$$

This ensures:

- consistent brightness scaling.
- no frame-to-frame variation.
- fair comparison between signals.

Output

Reusable function:

$$normalize_{s}ignal\left(x, min_{v}al, max_{v}al\right)$$

This will be used everywhere:

- BEV rendering.
- forward view.
- semantic overlay.

```
# Normalization Function

def normalize_signal(x, vmin, vmax):
    """
    Normalize signal to [0, 1] using fixed bounds.

    Args:
        x (np.ndarray): input signal
        vmin (float): lower bound (p1)
        vmax (float): upper bound (p99)

    Returns:
        np.ndarray: normalized signal in [0, 1]
    """
    eps = 1e-6

    x_clipped = np.clip(x, vmin, vmax)
    x_norm = (x_clipped - vmin) / (vmax - vmin + eps)

    return x_norm

# Sanity Check

# Apply normalization
I_norm = normalize_signal(I, INTENSITY_MIN, INTENSITY_MAX)
PR_norm = normalize_signal(PR, PROXY_MIN, PROXY_MAX)

print("Sanity Check:")
print("Intensity norm -> min:", I_norm.min(), "max:", I_norm.max())
print("Proxy norm     -> min:", PR_norm.min(), "max:", PR_norm.max())
```

```
Sanity Check:
Intensity norm -> min: 0.0 max: 0.99999857
Proxy norm     -> min: 0.0 max: 0.99999994
```

Build Bird's-Eye View (BEV) Renderer

We now construct the first visualization primitive: a BEV renderer.

Purpose

Convert LiDAR points into a top-down image:

(x, y) → image plane

Design Constraints

1. Fixed spatial limits (no auto-zoom).
2. Subsampling for clarity.
3. No density clutter.
4. Signal-driven coloring.

Mapping

- X-axis → horizontal.
- Y-axis → vertical.
- Color → signal value (intensity or proxy).

Spatial Limits (fixed)

We define a consistent region:

- $X \in [-50, 50]$ meters.
- $Y \in [-50, 50]$ meters.

This ensures:

- consistency across frames.
- comparable spatial context.

Output

Function: 'render_bev (xyz, signal_norm)'

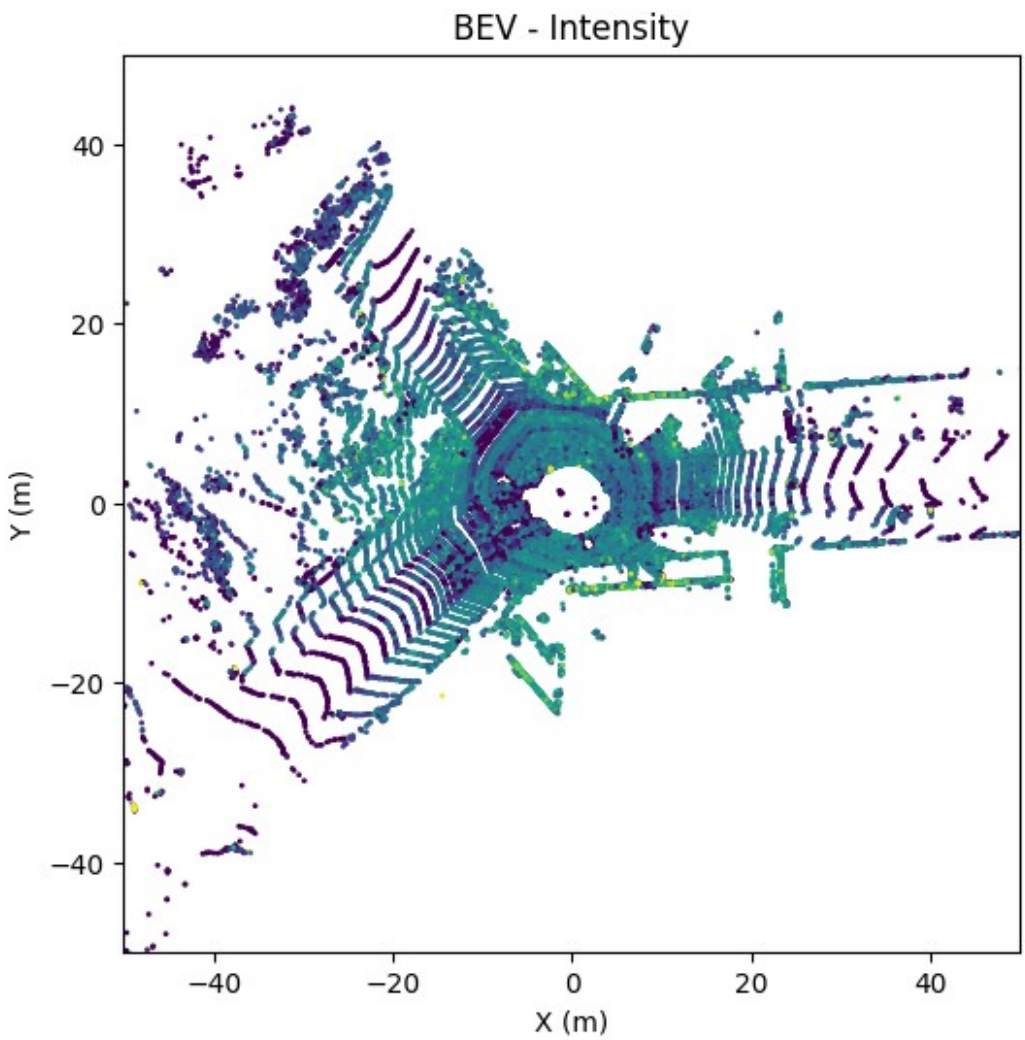
Returns a clean scatter-based BEV visualization.

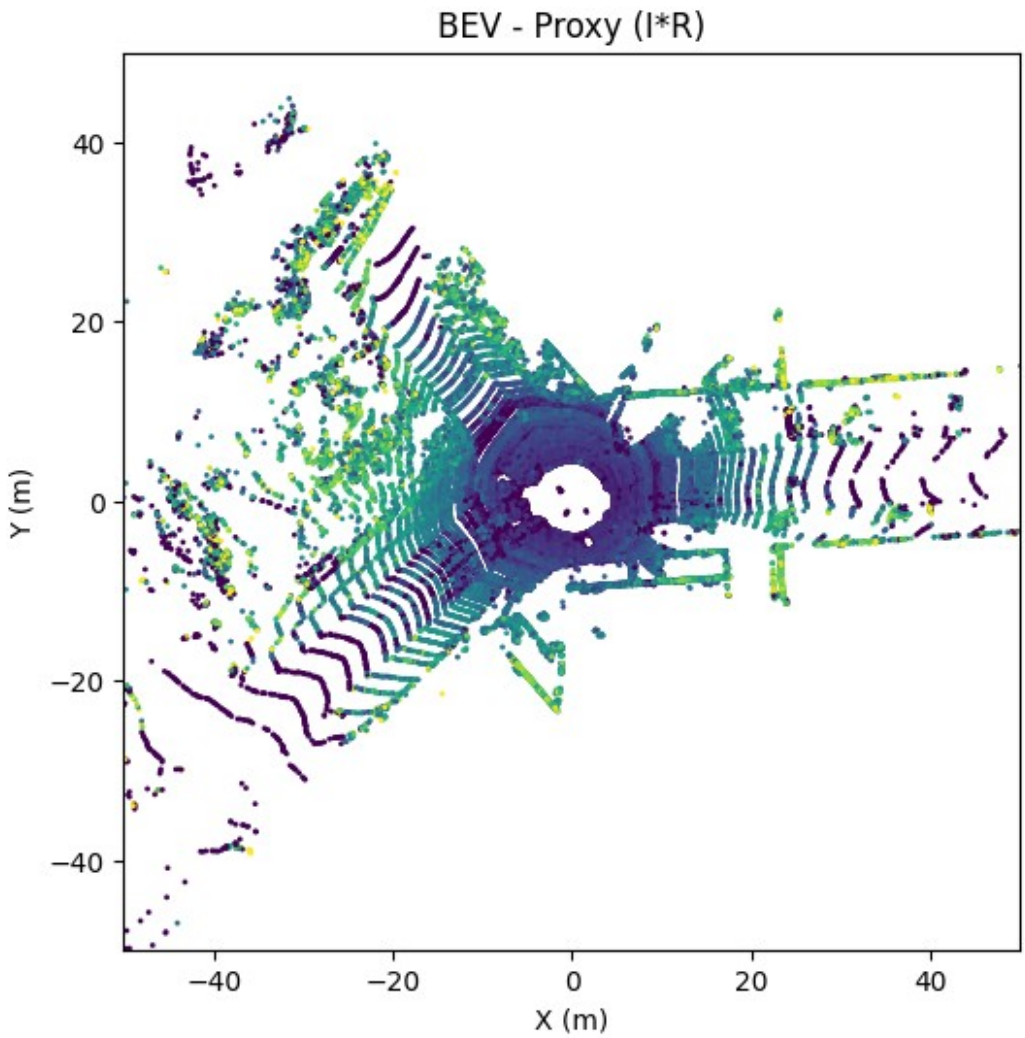
This is the base for:

- signal comparison board.
- semantic overlay (later).

```python
import matplotlib.pyplot as plt

# BEV Renderer

def render_bev(xyz, signal_norm, title="BEV View", max_points=80000):
    """
    Render Bird's-Eye View (top-down).

    Args:
        xyz (np.ndarray): (N, 3)
        signal_norm (np.ndarray): normalized signal [0,1]
        title (str): plot title
        max_points (int): subsampling limit
    """

    # Subsample for clarity
    N = xyz.shape[0]
    if N > max_points:
        idx = np.random.choice(N, max_points, replace=False)
        xyz = xyz[idx]
        signal_norm = signal_norm[idx]

    x = xyz[:, 0]
    y = xyz[:, 1]

    plt.figure(figsize=(6, 6))
    plt.scatter(
        x,
        y,
        c=signal_norm,
        s=1
    )

    # Fixed limits
    plt.xlim(-50, 50)
    plt.ylim(-50, 50)

    plt.xlabel("X (m)")
    plt.ylabel("Y (m)")
    plt.title(title)
    plt.gca().set_aspect('equal', adjustable='box')

    plt.show()

# Test BEV Rendering

# Use last loaded window data (from previous step)
# Need xyz again → quick reload (only one frame for test)

test_fid = selected_frame_ids[0]
```

```
bin_path = velodyne_dir / f"{test_fid}.bin"
scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)

xyz_test = scan[:, :3]
I_test = scan[:, 3]
R_test = np.linalg.norm(xyz_test, axis=1)
PR_test = I_test * R_test

# Normalize
I_test_norm = normalize_signal(I_test, INTENSITY_MIN, INTENSITY_MAX)
PR_test_norm = normalize_signal(PR_test, PROXY_MIN, PROXY_MAX)

# Render
render_bev(xyz_test, I_test_norm, title="BEV - Intensity")
render_bev(xyz_test, PR_test_norm, title="BEV - Proxy (I*R)")
```





Make BEV Renderer Deterministic and Video-Safe

The previous BEV renderer is visually correct, but not yet suitable for video/GIF generation.

Problems to Fix

1. **Random subsampling**
   - Causes frame-to-frame flicker.
   - Same region looks different every frame.
2. **Implicit colormap**
   - Not explicitly controlled.
   - Can lead to inconsistent visual interpretation.
3. **No color scaling lock**
   - Must explicitly enforce vmin=0, vmax=1.

Fixes Introduced

1. Deterministic Subsampling

Instead of random sampling:

- we take a fixed stride-based subset.
- ensures consistent point selection across frames.

2. Explicit Colormap

We use:

`viridis` (perceptually uniform, stable)

3. Fixed Color Mapping

We enforce:

- vmin = 0.
- vmax = 1.

Outcome

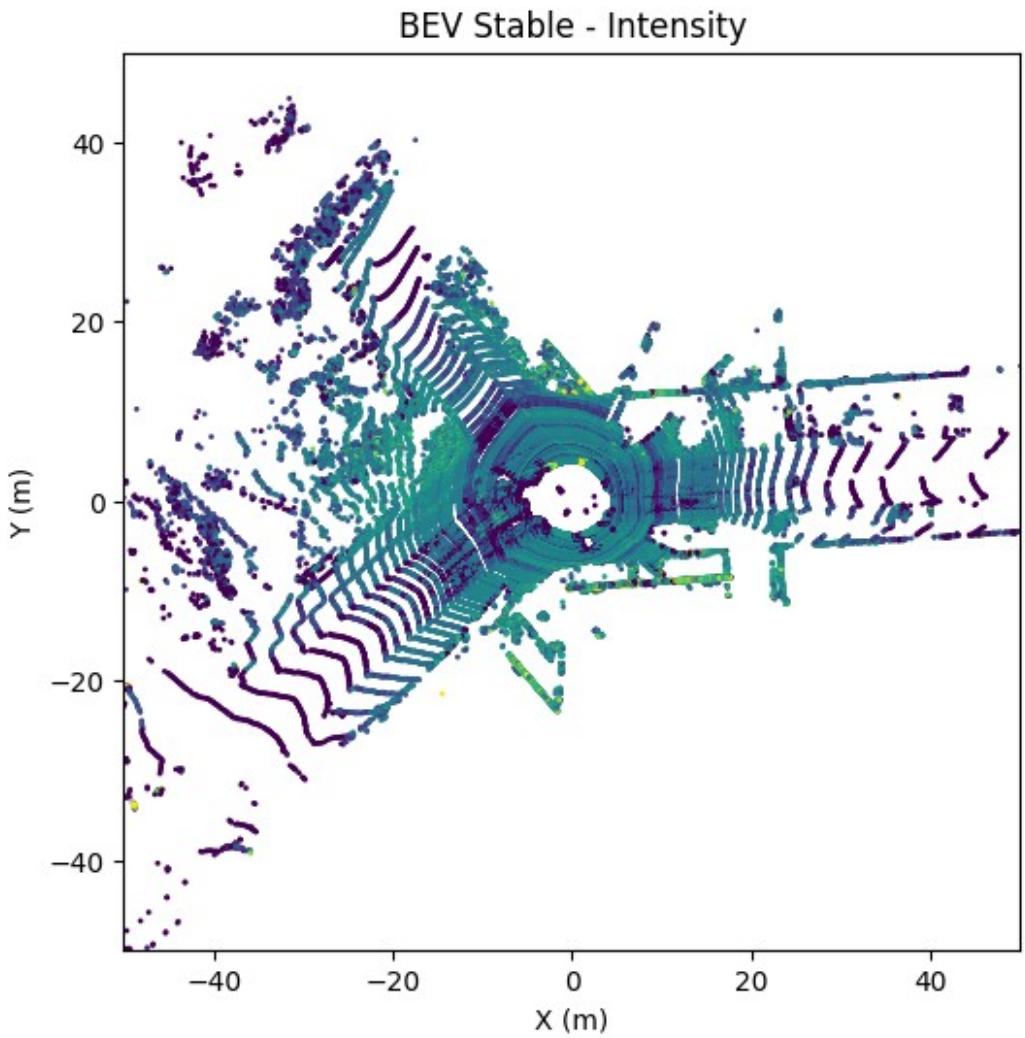
We now have a BEV renderer that is:

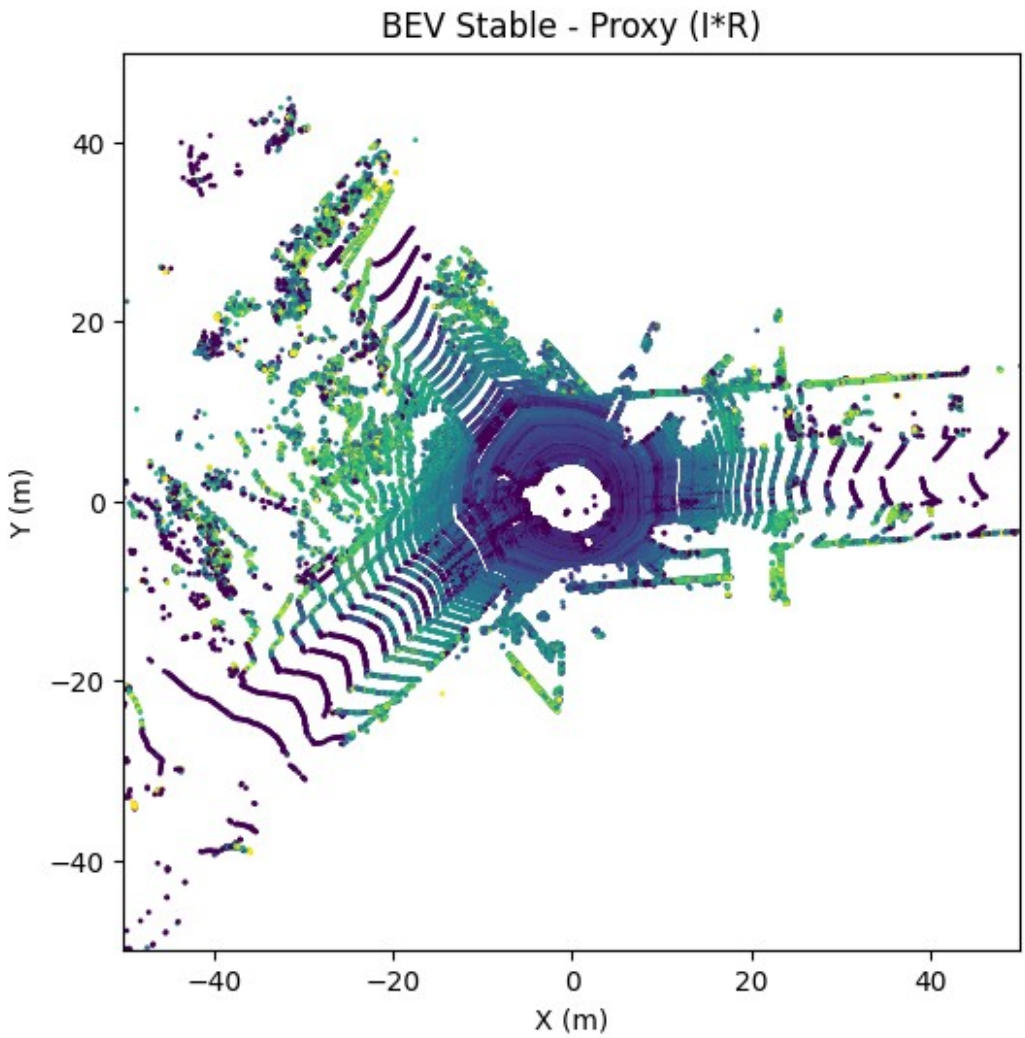
- stable across frames.
- consistent in color interpretation.
- safe for GIF/video generation.

This becomes a **core reusable primitive** for all later visualizations.

```
import matplotlib.pyplot as plt

# Deterministic Subsampling

def deterministic_subsample(xyz, signal, max_points=80000):
    N = xyz.shape[0]

    if N <= max_points:
        return xyz, signal

    stride = N // max_points
    idx = np.arange(0, N, stride)

    return xyz[idx], signal[idx]

# Stable BEV Renderer

def render_bev_stable(xyz, signal_norm, title="BEV View"):
```

```
    xyz_sub, signal_sub = deterministic_subsample(xyz, signal_norm)

    x = xyz_sub[:, 0]
    y = xyz_sub[:, 1]

    plt.figure(figsize=(6, 6))

    plt.scatter(
        x,
        y,
        c=signal_sub,
        s=1,
        cmap="viridis",
        vmin=0.0,
        vmax=1.0
    )

    plt.xlim(-50, 50)
    plt.ylim(-50, 50)

    plt.xlabel("X (m)")
    plt.ylabel("Y (m)")
    plt.title(title)

    plt.gca().set_aspect('equal', adjustable='box')

    plt.show()

# Test Stable Renderer

render_bev_stable(xyz_test, I_test_norm, title="BEV Stable -
Intensity")
render_bev_stable(xyz_test, PR_test_norm, title="BEV Stable - Proxy
(I*R)")
```


BEV Stable - Intensity
40
20
0
−20
−40
Y (m)
−40
−20
0
20
40
X (m)



Build Forward View (FOV) Renderer

We now construct the second core visualization primitive: the Forward View (FOV).

Purpose

Unlike BEV, which shows spatial layout, the forward view reveals:

- LiDAR scan structure (ring pattern).
- Range progression along rays.
- Vertical geometry (height variation).
- Signal variation along sensing direction..

This is critical for interpreting:

how intensity and pseudo-reflectivity behave with respect to range

Projection Model

We use a simple geometric projection:

- X → forward distance.
- Z → height.

So each point: (x, y, z) → (x, z)

Why NOT perspective projection?

We deliberately avoid camera-style projection because:

- this is LiDAR signal visualization, not image rendering.
- we want **raw structure visibility**, not realism.
- consistency > visual aesthetics.

Rendering Design

We apply the same principles as BEV:

- fixed spatial limits.
- deterministic subsampling.
- consistent colormap (viridis).
- fixed normalization [0,1].

Spatial Limits (fixed)

We define:

- $X \in [0, 50]$ meters (forward only).
- $Z \in [-3, 3]$ meters (height band).

This ensures:

- consistent frame alignment.
- interpretable vertical structure.
- comparable across frames.

Output

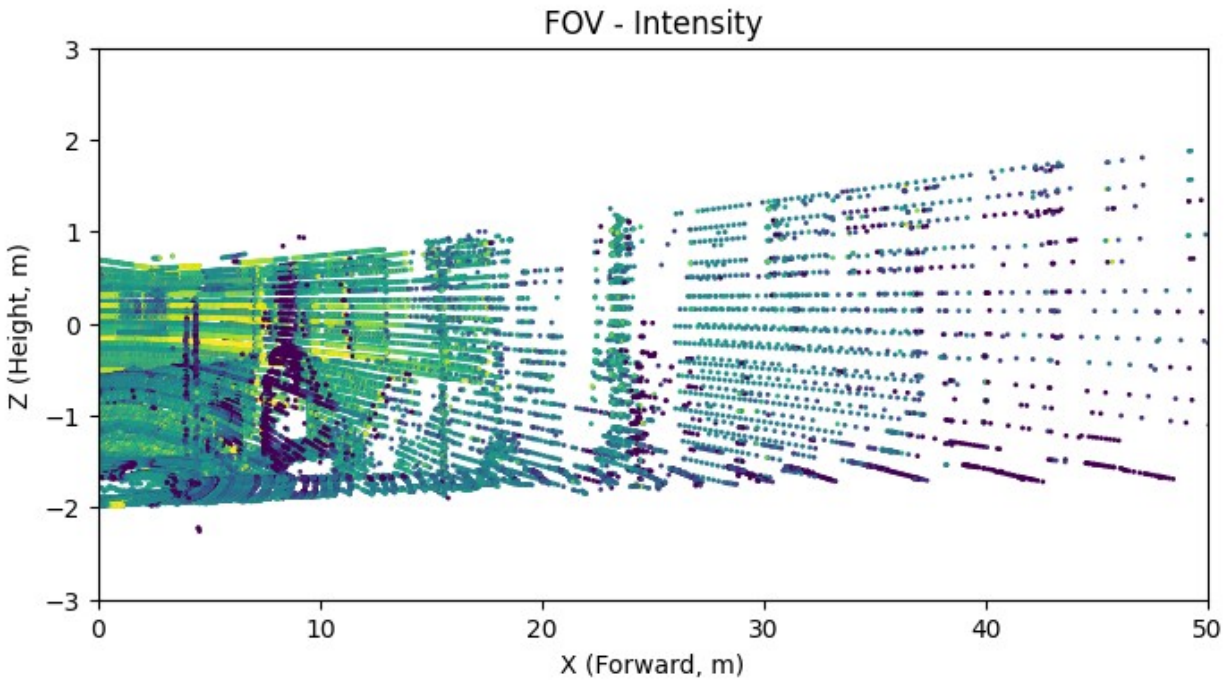
Function:

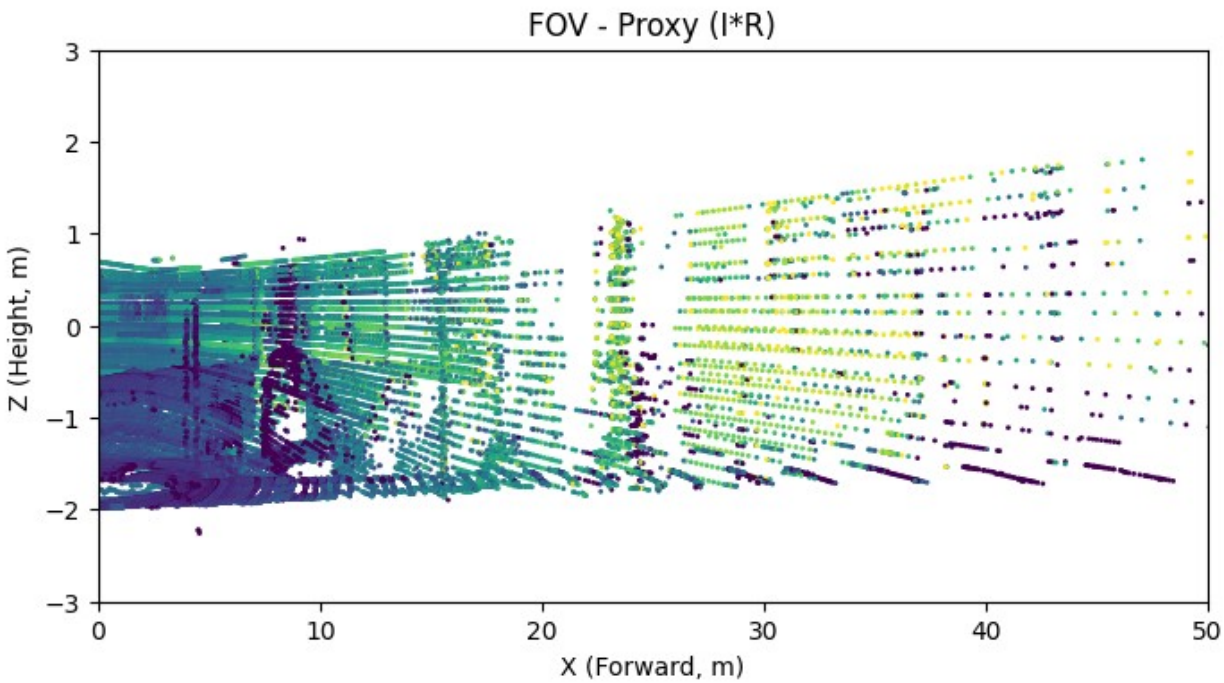
render_fov(xyz, signal_norm)

This becomes the second fundamental building block for:

- signal comparison boards.
- semantic overlay.
- video generation.

```python
# Stable Forward View Renderer

def render_fov(xyz, signal_norm, title="Forward View"):

    xyz_sub, signal_sub = deterministic_subsample(xyz, signal_norm)

    x = xyz_sub[:, 0]   # forward
    z = xyz_sub[:, 2]   # height

    plt.figure(figsize=(8, 4))

    plt.scatter(
        x,
        z,
        c=signal_sub,
        s=1,
        cmap="viridis",
        vmin=0.0,
        vmax=1.0
    )

    # Fixed limits
    plt.xlim(0, 50)
    plt.ylim(-3, 3)

    plt.xlabel("X (Forward, m)")
    plt.ylabel("Z (Height, m)")
    plt.title(title)

    plt.show()

# Test FOV Rendering

render_fov(xyz_test, I_test_norm, title="FOV - Intensity")
render_fov(xyz_test, PR_test_norm, title="FOV - Proxy (I*R)")
```





Build Signal Comparison Board (BEV + FOV)

We now combine the rendering primitives into a structured visualization board.

Purpose

Create a **clean, interpretable comparison figure** showing:

- geometry (structure only).
- raw intensity.
- pseudo-reflectivity (I · R).

across:

- BEV (top-down).
- Forward View (sensor perspective).

Design Principles

1. Same spatial limits across panels.
2. Same normalization across frames.
3. Same colormap for signal panels.
4. Geometry shown in neutral gray.
5. Clean layout (no clutter).

Geometry Rendering

For geometry panels:

- no signal coloring.
- just point structure.
- light gray color.

This isolates:

> pure spatial structure vs signal effects

Output

Function:

render_signal_board(xyz, I_norm, PR_norm)

Produces a publication-style comparison board.

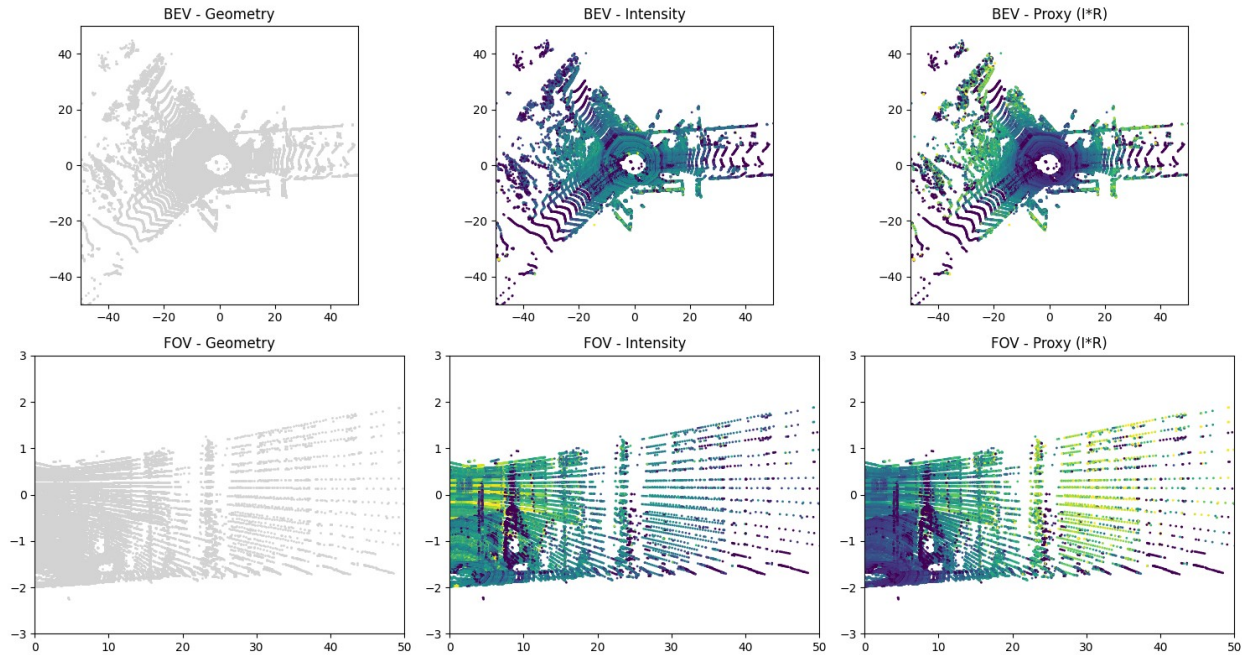
This will later be extended into:

- frame sequences.
- GIFs.
- report figures.

```
# Geometry Renderer (shared)

def render_geometry(ax, xyz, max_points=80000):
    xyz_sub, _ = deterministic_subsample(xyz, np.zeros(len(xyz)),
```

```
max_points)

    x = xyz_sub[:, 0]
    y = xyz_sub[:, 1]
    z = xyz_sub[:, 2]

    ax.scatter(x, y, s=1, c='lightgray')
    ax.set_xlim(-50, 50)
    ax.set_ylim(-50, 50)
    ax.set_title("BEV - Geometry")
    ax.set_aspect('equal')


def render_geometry_fov(ax, xyz, max_points=80000):
    xyz_sub, _ = deterministic_subsample(xyz, np.zeros(len(xyz)),
max_points)

    x = xyz_sub[:, 0]
    z = xyz_sub[:, 2]

    ax.scatter(x, z, s=1, c='lightgray')
    ax.set_xlim(0, 50)
    ax.set_ylim(-3, 3)
    ax.set_title("FOV - Geometry")

# Signal Board Renderer

def render_signal_board(xyz, I_norm, PR_norm):

    xyz_sub, I_sub = deterministic_subsample(xyz, I_norm)
    _, PR_sub = deterministic_subsample(xyz, PR_norm)

    fig, axes = plt.subplots(2, 3, figsize=(15, 8))

    # BEV Row

    # Geometry
    render_geometry(axes[0, 0], xyz)

    # Intensity
    axes[0, 1].scatter(
        xyz_sub[:, 0], xyz_sub[:, 1],
        c=I_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[0, 1].set_xlim(-50, 50)
    axes[0, 1].set_ylim(-50, 50)
    axes[0, 1].set_title("BEV - Intensity")
    axes[0, 1].set_aspect('equal')
```

```
    # Proxy
    axes[0, 2].scatter(
        xyz_sub[:, 0], xyz_sub[:, 1],
        c=PR_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[0, 2].set_xlim(-50, 50)
    axes[0, 2].set_ylim(-50, 50)
    axes[0, 2].set_title("BEV - Proxy (I*R)")
    axes[0, 2].set_aspect('equal')


    # FOV Row

    # Geometry
    render_geometry_fov(axes[1, 0], xyz)

    # Intensity
    axes[1, 1].scatter(
        xyz_sub[:, 0], xyz_sub[:, 2],
        c=I_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[1, 1].set_xlim(0, 50)
    axes[1, 1].set_ylim(-3, 3)
    axes[1, 1].set_title("FOV - Intensity")

    # Proxy
    axes[1, 2].scatter(
        xyz_sub[:, 0], xyz_sub[:, 2],
        c=PR_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[1, 2].set_xlim(0, 50)
    axes[1, 2].set_ylim(-3, 3)
    axes[1, 2].set_title("FOV - Proxy (I*R)")

    plt.tight_layout()
    plt.show()

# Test Board

render_signal_board(xyz_test, I_test_norm, PR_test_norm)
```



Refined Signal Board (Alignment + Clarity Fixes)

We now correct structural issues in the signal comparison board.

Fixes Applied

1. Single shared subsampling:
   - ensures identical points across all panels.
2. Improved geometry visibility:
   - darker neutral gray.
   - slightly higher opacity.
3. Cleaner layout:
   - better spacing.
   - reduced visual clutter.
4. Consistent rendering hierarchy:
   - geometry = context.
   - signal = focus.

Goal

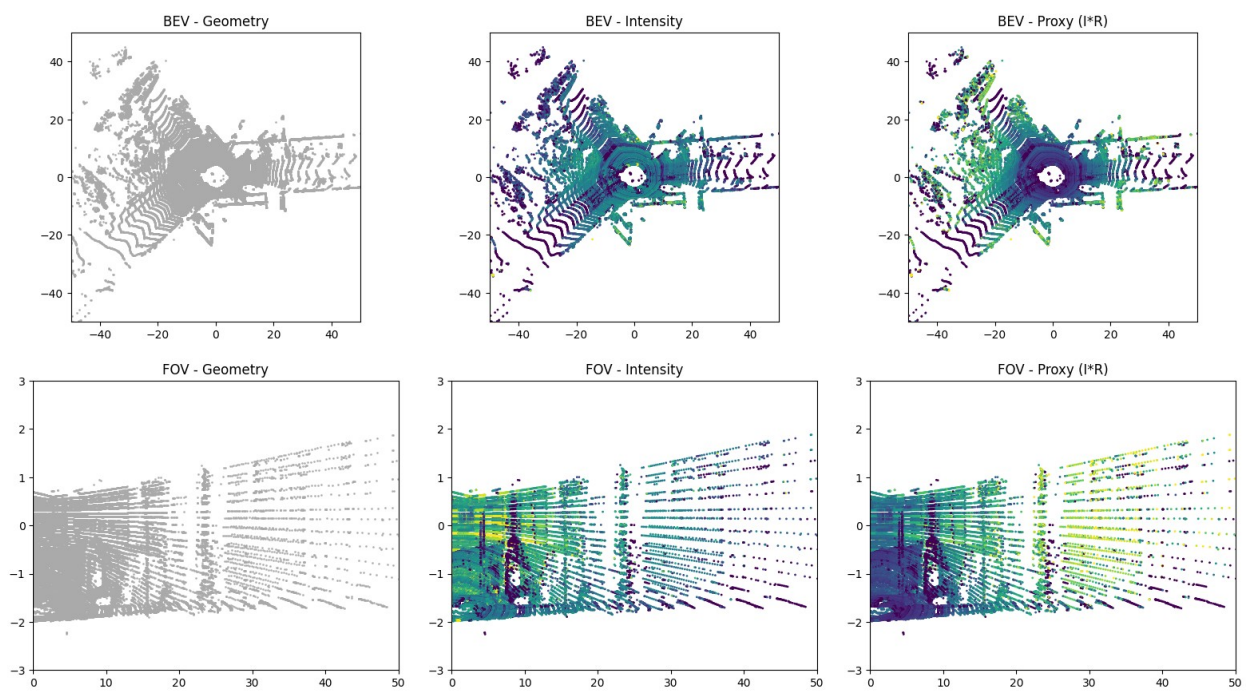
Transform the board from: "correct but rough"

into: "clean, aligned, publication-ready"

```
def render_signal_board_clean(xyz, I_norm, PR_norm):

    # SINGLE SHARED SUBSAMPLING
```

```
    xyz_sub, I_sub = deterministic_subsample(xyz, I_norm)
    _, PR_sub = xyz_sub, PR_norm[:len(xyz_sub)]  # enforce alignment

    fig, axes = plt.subplots(2, 3, figsize=(16, 9))

    # BEV ROW

    # Geometry (darker + visible)
    axes[0, 0].scatter(
        xyz_sub[:, 0], xyz_sub[:, 1],
        s=1, c='#aaaaaa', alpha=0.8
    )
    axes[0, 0].set_xlim(-50, 50)
    axes[0, 0].set_ylim(-50, 50)
    axes[0, 0].set_title("BEV - Geometry")
    axes[0, 0].set_aspect('equal')

    # Intensity
    axes[0, 1].scatter(
        xyz_sub[:, 0], xyz_sub[:, 1],
        c=I_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[0, 1].set_xlim(-50, 50)
    axes[0, 1].set_ylim(-50, 50)
    axes[0, 1].set_title("BEV - Intensity")
    axes[0, 1].set_aspect('equal')

    # Proxy
    axes[0, 2].scatter(
        xyz_sub[:, 0], xyz_sub[:, 1],
        c=PR_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[0, 2].set_xlim(-50, 50)
    axes[0, 2].set_ylim(-50, 50)
    axes[0, 2].set_title("BEV - Proxy (I*R)")
    axes[0, 2].set_aspect('equal')

    # FOV ROW

    # Geometry
    axes[1, 0].scatter(
        xyz_sub[:, 0], xyz_sub[:, 2],
        s=1, c='#aaaaaa', alpha=0.8
    )
    axes[1, 0].set_xlim(0, 50)
    axes[1, 0].set_ylim(-3, 3)
    axes[1, 0].set_title("FOV - Geometry")
```

```
    # Intensity
    axes[1, 1].scatter(
        xyz_sub[:, 0], xyz_sub[:, 2],
        c=I_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[1, 1].set_xlim(0, 50)
    axes[1, 1].set_ylim(-3, 3)
    axes[1, 1].set_title("FOV - Intensity")

    # Proxy
    axes[1, 2].scatter(
        xyz_sub[:, 0], xyz_sub[:, 2],
        c=PR_sub, s=1,
        cmap="viridis", vmin=0, vmax=1
    )
    axes[1, 2].set_xlim(0, 50)
    axes[1, 2].set_ylim(-3, 3)
    axes[1, 2].set_title("FOV - Proxy (I*R)")

    plt.tight_layout(pad=2.0)
    plt.show()


# TEST CLEAN VERSION
render_signal_board_clean(xyz_test, I_test_norm, PR_test_norm)
```



Semantic Reflectivity Overlay (Correct & Aligned)

We now rebuild the semantic overlay with correct point-label alignment.

Critical Fix

Previous version had a silent issue:

- subsampling was applied to xyz.
- but semantic labels were sliced independently.

This breaks:

point ↔ label correspondence

Correct Approach

We enforce:

- one shared index set.
- applied to:
  - xyz.
  - proxy.
  - semantic labels.

So:

(xyz_sub, proxy_sub, labels_sub) are all aligned point-wise.

Visualization

- Color → semantic class.
- Brightness → proxy ($\hat{\rho}$).

Final color: RGB_final = RGB_class × brightness

Output

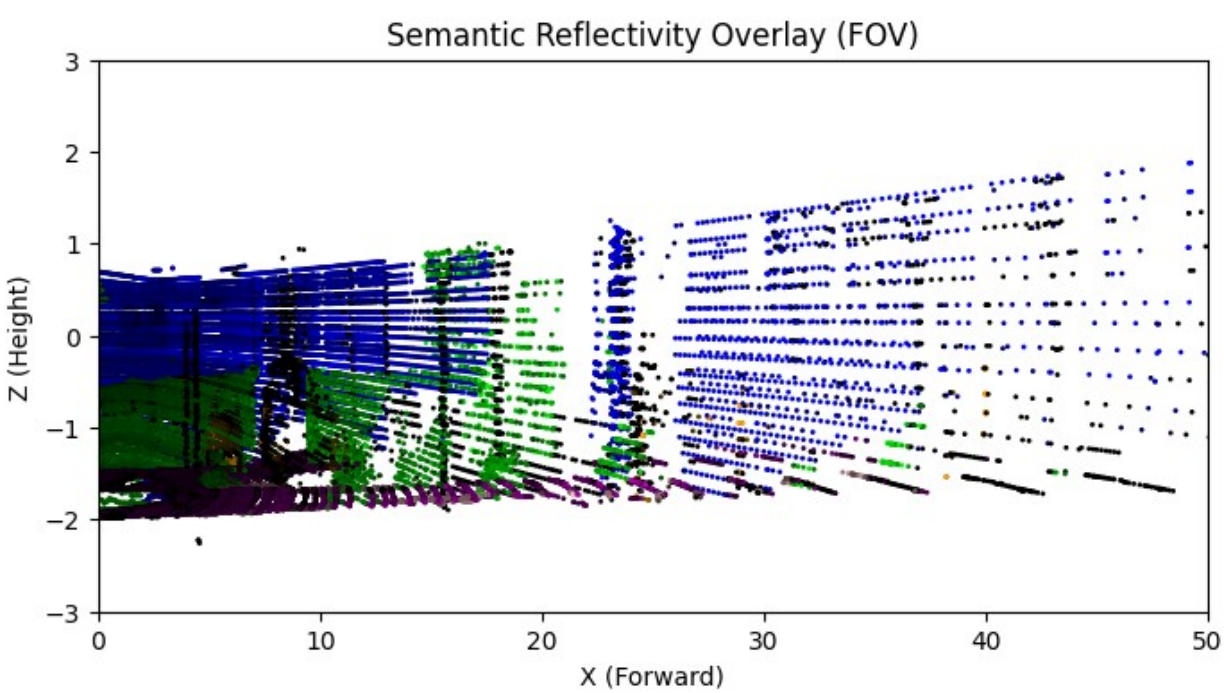
Correct, aligned semantic overlay:

render_semantic_overlay_fov(...)

```
# Semantic Color Map

SEMANTIC_COLOR_MAP = {
    40: (255, 0, 255),     # road
    48: (128, 0, 128),     # sidewalk
    70: (0, 255, 0),       # vegetation
    50: (0, 0, 255),       # building
```

```
    10: (255, 165, 0),     # car
    44: (255, 192, 203),   # parking
}

def map_semantic_to_color(labels):
    colors = np.zeros((len(labels), 3))

    for cls, rgb in SEMANTIC_COLOR_MAP.items():
        mask = labels == cls
        colors[mask] = np.array(rgb) / 255.0

    return colors

# Deterministic Subsampling (returns indices)

def deterministic_subsample_indices(N, max_points=80000):
    if N <= max_points:
        return np.arange(N)

    stride = N // max_points
    return np.arange(0, N, stride)

# Semantic Overlay Renderer (FOV)

def render_semantic_overlay_fov(xyz, semantic_labels, proxy_norm,
title="Semantic Overlay (FOV)"):

    # --- aligned subsampling ---
    N = xyz.shape[0]
    idx = deterministic_subsample_indices(N)

    xyz_sub = xyz[idx]
    proxy_sub = proxy_norm[idx]
    labels_sub = semantic_labels[idx]

    # --- color mapping ---
    base_colors = map_semantic_to_color(labels_sub)

    # --- brightness modulation ---
    final_colors = base_colors * proxy_sub[:, None]

    x = xyz_sub[:, 0]
    z = xyz_sub[:, 2]

    plt.figure(figsize=(8, 4))

    plt.scatter(
        x,
        z,
        c=final_colors,
```

```
        s=1
    )

    plt.xlim(0, 50)
    plt.ylim(-3, 3)

    plt.title(title)
    plt.xlabel("X (Forward)")
    plt.ylabel("Z (Height)")

    plt.show()

# Resolve labels dir
labels_dir = DATASET_ROOT / "labels"

# Load test frame labels
label_path = labels_dir / f"{test_fid}.label"
labels_test = np.fromfile(label_path, dtype=np.uint32)
semantic_test = labels_test & 0xFFFF

# Render overlay
render_semantic_overlay_fov(
    xyz_test,
    semantic_test,
    PR_test_norm,
    title="Semantic Reflectivity Overlay (FOV)"
)
```



Interpretation: Semantic Reflectivity Overlay (FOV)

This visualization combines semantic labels with pseudo-reflectivity:

- **Color → semantic class**.
- **Brightness → proxy signal ($\hat{\rho} = I \cdot R$)**.

Semantic Color Mapping (Reference)

The following color scheme is used for semantic classes:

- **Magenta (255, 0, 255)** → Road (40).
- **Purple (128, 0, 128)** → Sidewalk (48).
- **Green (0, 255, 0)** → Vegetation (70).
- **Blue (0, 0, 255)** → Building (50).
- **Orange (255, 165, 0)** → Car (10).
- **Pink (255, 192, 203)** → Parking (44).

Note

- Only major classes are visualized for clarity.
- Other classes are either omitted or appear as low-intensity points.
- Brightness variation within each color is driven by the proxy signal ($\hat{\rho}$), not by class changes.

Key Observations

1. **Class identity is preserved**:
   - Each semantic class remains visually distinguishable.
   - Brightness modulation does not overwrite class color.
2. **Within-class variation becomes visible**:
   - Different points within the same class (e.g., vegetation, road) exhibit varying brightness.
   - This reflects underlying signal differences not captured by labels alone.
3. **Improved visibility at longer ranges**:
   - Compared to raw intensity, far-range structures are more visible.
   - Proxy reduces distance-induced attenuation effects.
4. **Scan structure remains intact**:
   - LiDAR ring patterns are preserved.
   - No artificial smoothing or distortion is introduced.

Interpretation

This overlay shows that pseudo-reflectivity:

- does not replace semantic labels.
- but provides an additional signal layer.
- that can reveal intra-class structure and range-aware variation.

This supports the project's core idea:

> reflectivity-aware augmentation can enhance interpretability.
> without claiming calibrated reflectivity recovery.

Semantic Overlay Board (FOV + Legend + Class Stats)

We now upgrade the semantic overlay into a structured visualization board.

Purpose

Move from:

> single overlay plot

to:

> a **self-contained visualization artifact** that includes:

- semantic overlay (FOV).
- class legend.
- class distribution summary.

Layout

Left (Main)	Right (Info Panel)
Semantic Overlay (FOV)	Legend + Class Stats

Components

1. Semantic Overlay (FOV)

- Color → class.
- Brightness → proxy.

2. Legend Panel

- Maps color → semantic class.
- Ensures interpretability.

3. Class Distribution

- Shows dominant classes in current frame.
- Provides context for visual density.

Design Principles

- clean separation of visualization and information.
- minimal clutter.
- readable without external explanation.
- consistent with project honesty constraints.

Output

render_semantic_board(...)

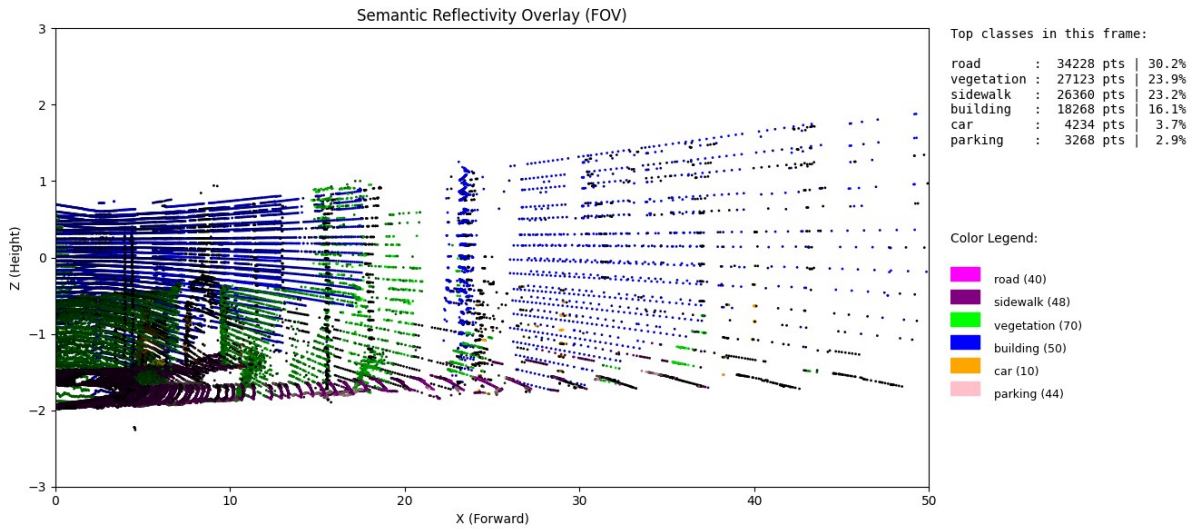
This will be used for:

- report figures.
- final GIF frames.

```
from collections import Counter

# Class Name Mapping (subset)

CLASS_NAME_MAP = {
    40: "road",
    48: "sidewalk",
    70: "vegetation",
    50: "building",
    10: "car",
    44: "parking",
}

# Semantic Board Renderer

def render_semantic_board(xyz, semantic_labels, proxy_norm):

    # aligned subsampling
    N = xyz.shape[0]
    idx = deterministic_subsample_indices(N)

    xyz_sub = xyz[idx]
    proxy_sub = proxy_norm[idx]
    labels_sub = semantic_labels[idx]

    # color mapping
    base_colors = map_semantic_to_color(labels_sub)
    final_colors = base_colors * proxy_sub[:, None]

    # layout
```

```python
    fig = plt.figure(figsize=(14, 6))
    gs = fig.add_gridspec(1, 2, width_ratios=[3, 1])

    # Left: FOV Overlay

    ax_main = fig.add_subplot(gs[0, 0])

    ax_main.scatter(
        xyz_sub[:, 0],
        xyz_sub[:, 2],
        c=final_colors,
        s=1
    )

    ax_main.set_xlim(0, 50)
    ax_main.set_ylim(-3, 3)

    ax_main.set_title("Semantic Reflectivity Overlay (FOV)")
    ax_main.set_xlabel("X (Forward)")
    ax_main.set_ylabel("Z (Height)")

    # Right: Legend + Stats

    ax_info = fig.add_subplot(gs[0, 1])
    ax_info.axis('off')

    # Class counts
    counts = Counter(labels_sub)

    # Filter only mapped classes
    filtered = {
        cls: cnt for cls, cnt in counts.items()
        if cls in CLASS_NAME_MAP
    }

    # Sort by count
    sorted_classes = sorted(filtered.items(), key=lambda x: -x[1])

    total = sum(filtered.values())

    # Text block
    text_lines = []
    text_lines.append("Top classes in this frame:\n")

    for cls, cnt in sorted_classes[:6]:
        name = CLASS_NAME_MAP[cls]
        pct = (cnt / total) * 100
        text_lines.append(f"{name:10} : {cnt:6d} pts | {pct:4.1f}%")
```

```
    text_block = "\n".join(text_lines)

    ax_info.text(0, 1, text_block, va='top', fontsize=10,
family='monospace')

    # Legend
    y_offset = 0.5
    ax_info.text(0, y_offset, "\nColor Legend:\n", fontsize=10)

    for i, (cls, name) in enumerate(CLASS_NAME_MAP.items()):
        rgb = np.array(SEMANTIC_COLOR_MAP[cls]) / 255.0

        ax_info.add_patch(
            plt.Rectangle((0, y_offset - 0.05*(i+1)), 0.1, 0.03,
color=rgb)
        )

        ax_info.text(
            0.15,
            y_offset - 0.05*(i+1),
            f"{name} ({cls})",
            va='center',
            fontsize=9
        )

    plt.tight_layout()
    plt.show()

# TEST BOARD

render_semantic_board(
    xyz_test,
    semantic_test,
    PR_test_norm
)
```



Frame Generator for Visualization Sequences

We now build the core mechanism for generating frame-wise visual outputs.

Purpose

Convert:

> static rendering functions

into:

> a sequence of consistent frames across time.

This is required for:

- GIF generation.
- video export.
- temporal visualization of proxy behavior.

Key Idea

For each frame in a selected window:

1. Load LiDAR scan.
2. Compute:
   - intensity (I).
   - range (R).
   - proxy ($\hat{\rho} = I \cdot R$).
3. Normalize using fixed global bounds.
4. Render visualization.
5. Save frame as image.

Constraints

1. **No per-frame normalization**.
2. **Fixed spatial limits**.
3. **Deterministic sampling**.
4. **Consistent rendering functions**.

Output

We will build:

generate_frames(window_id, mode)

Where:

- mode = "signal" or "semantic".
- output = sequence of saved PNG frames.

Next Step

These frames will be used to:

- create GIFs.
- create MP4 videos.

This is the backbone of final artifacts.

```
import os
from tqdm import tqdm

# Output directory

FRAME_DIR = Path("../artifacts/frames")
FRAME_DIR.mkdir(parents=True, exist_ok=True)

# Frame Generator

def generate_frames(window_id, mode="signal", max_frames=30):

    # load window metadata
    row = window_df[window_df["window_id"] == window_id].iloc[0]

    start_idx = int(row["start_idx"])
    end_idx = int(row["end_idx"])

    frame_ids = sorted([f.stem for f in velodyne_dir.glob("*.bin")])
    selected_ids = frame_ids[start_idx:end_idx + 1][:max_frames]

    print(f"Generating frames for window: {window_id}")
    print(f"Total frames: {len(selected_ids)}")
```

```python
    # loop
    for i, fid in enumerate(tqdm(selected_ids)):

        # Load scan
        bin_path = velodyne_dir / f"{fid}.bin"
        scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)

        xyz = scan[:, :3]
        I = scan[:, 3]
        R = np.linalg.norm(xyz, axis=1)
        PR = I * R

        # Normalize
        I_norm = normalize_signal(I, INTENSITY_MIN, INTENSITY_MAX)
        PR_norm = normalize_signal(PR, PROXY_MIN, PROXY_MAX)

        # Rendering

        if mode == "signal":
            fig, ax = plt.subplots(1, 1, figsize=(6, 6))

            xyz_sub, PR_sub = deterministic_subsample(xyz, PR_norm)

            ax.scatter(
                xyz_sub[:, 0],
                xyz_sub[:, 1],
                c=PR_sub,
                s=1,
                cmap="viridis",
                vmin=0,
                vmax=1
            )

            ax.set_xlim(-50, 50)
            ax.set_ylim(-50, 50)
            ax.set_title(f"Frame {fid}")
            ax.set_aspect('equal')

        elif mode == "semantic":
            # Load labels
            label_path = labels_dir / f"{fid}.label"
            labels = np.fromfile(label_path, dtype=np.uint32)
            semantic = labels & 0xFFFF

            # Aligned subsampling
            idx = deterministic_subsample_indices(len(xyz))

            xyz_sub = xyz[idx]
```

```
            PR_sub = PR_norm[idx]
            labels_sub = semantic[idx]

            colors = map_semantic_to_color(labels_sub)
            final_colors = colors * PR_sub[:, None]

            fig, ax = plt.subplots(1, 1, figsize=(8, 4))

            ax.scatter(
                xyz_sub[:, 0],
                xyz_sub[:, 2],
                c=final_colors,
                s=1
            )

            ax.set_xlim(0, 50)
            ax.set_ylim(-3, 3)
            ax.set_title(f"Frame {fid}")

        # Save frame

        frame_path = FRAME_DIR / f"{mode}_{window_id}_{i:03d}.png"
        plt.savefig(frame_path, dpi=150)
        plt.close(fig)

    print("Frame generation complete.")

# TEST (small run)

generate_frames("medium_0", mode="semantic", max_frames=10)
```

```
Generating frames for window: medium_0
Total frames: 10

100%|
|| 10/10 [00:12<00:00,  1.26s/it]

Frame generation complete.
```

GIF Builder (Final Artifact Generation)

We now convert generated frames into animated GIFs.

Purpose

Transform:

static frame sequences

into:

compact, shareable visual artifacts

Why GIF?

- lightweight preview of results.
- easy to embed in reports / README.
- communicates temporal behavior clearly.

Input

frames generated in: **artifacts/frames/**

Output

GIF files stored in: **artifacts/gifs/**

Design Choices

- fixed frame order.
- consistent duration between frames.
- no interpolation (preserve raw behavior).

Result

A clean animation showing:

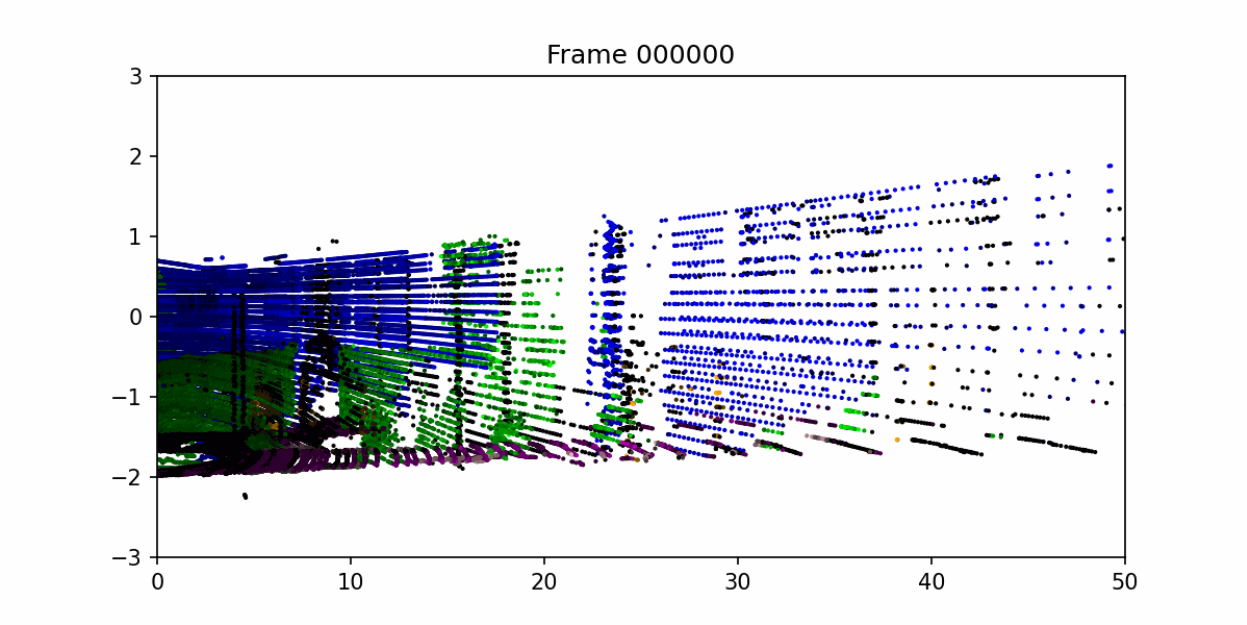
- temporal stability.
- proxy behavior across motion.
- semantic variation over time.

```
import imageio.v2 as imageio

# Output directory

GIF_DIR = Path("../artifacts/gifs")
GIF_DIR.mkdir(parents=True, exist_ok=True)

# GIF Builder

def build_gif(window_id, mode="semantic", fps=5):

    # Find frames
    frame_files = sorted(
        FRAME_DIR.glob(f"{mode}_{window_id}_*.png")
    )

    if len(frame_files) == 0:
```

```
        print("No frames found.")
        return

    print(f"Building GIF from {len(frame_files)} frames...")

    images = []

    for f in frame_files:
        img = imageio.imread(f)
        images.append(img)

    gif_path = GIF_DIR / f"{mode}_{window_id}.gif"

    imageio.mimsave(
        gif_path,
        images,
        duration=1.0 / fps
    )

    print(f"GIF saved to: {gif_path}")

# TEST GIF

build_gif("medium_0", mode="semantic", fps=5)
```

```
Building GIF from 10 frames...
GIF saved to: ../artifacts/gifs/semantic_medium_0.gif
```

```
from IPython.display import Image, display

# Path to generated GIF
gif_path = GIF_DIR / "semantic_medium_0.gif"

# Display
display(Image(filename=str(gif_path)))
```



Semantic Reflectivity Overlay (BEV)

We extend the semantic overlay to the BEV representation.

Purpose

Complement the forward view with:

- spatial layout (BEV).
- semantic structure.
- proxy modulation.

Representation

- Color → semantic class.
- Brightness → proxy ($\hat{\rho} = I \cdot R$).

Why BEV Matters Here

FOV shows:

- scan geometry.
- range effects.

BEV shows:

- spatial relationships.
- class distribution across scene.

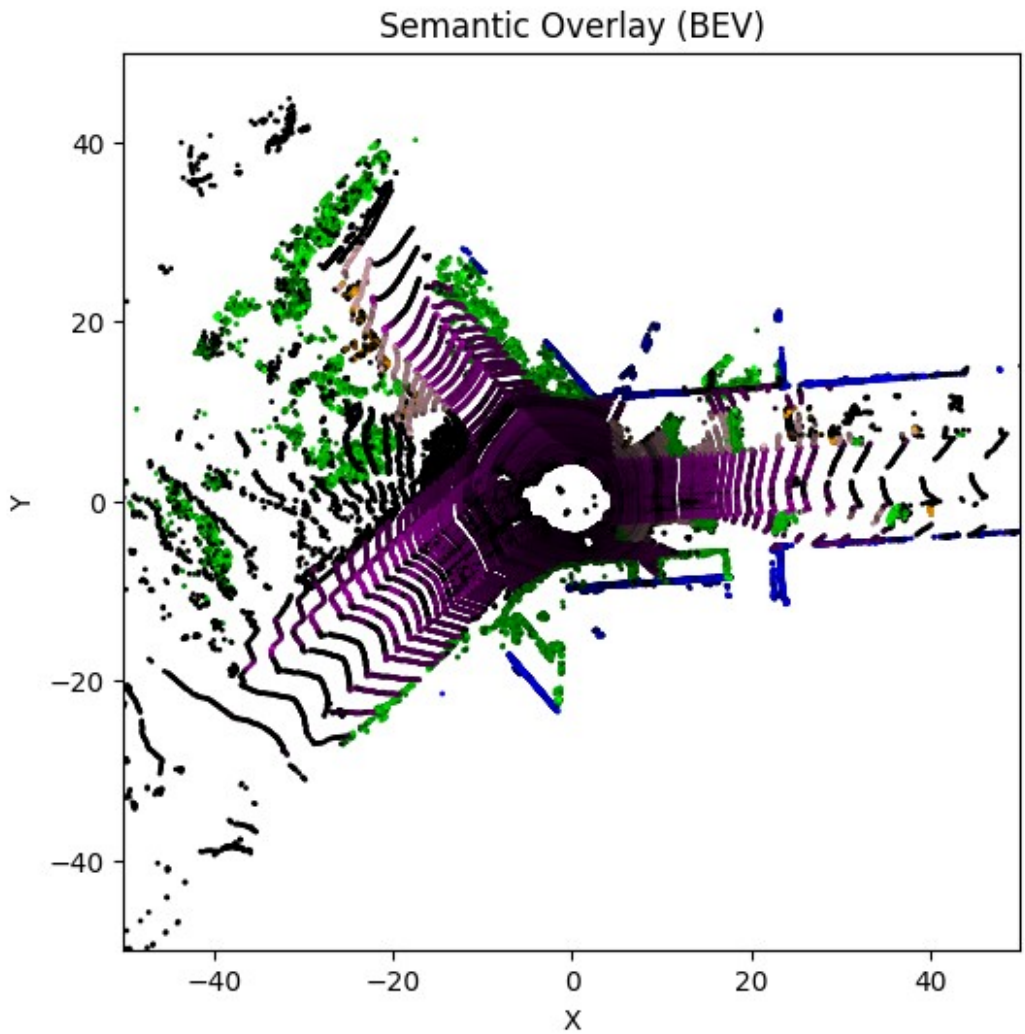
Together:

BEV + FOV provide a complete interpretability pair

Output

render_semantic_overlay_bev(...)

```
def render_semantic_overlay_bev(xyz, semantic_labels, proxy_norm,
title="Semantic Overlay (BEV)"):

    idx = deterministic_subsample_indices(len(xyz))

    xyz_sub = xyz[idx]
    proxy_sub = proxy_norm[idx]
    labels_sub = semantic_labels[idx]

    base_colors = map_semantic_to_color(labels_sub)
    final_colors = base_colors * proxy_sub[:, None]

    plt.figure(figsize=(6, 6))

    plt.scatter(
        xyz_sub[:, 0],
        xyz_sub[:, 1],
        c=final_colors,
        s=1
    )

    plt.xlim(-50, 50)
    plt.ylim(-50, 50)

    plt.title(title)
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.gca().set_aspect('equal')

    plt.show()


# TEST
render_semantic_overlay_bev(xyz_test, semantic_test, PR_test_norm)
```



Final Semantic Visualization Board

We now build the final polished artifact.

Layout

Left	Center	Right
BEV Overlay	FOV Overlay	Legend + Stats

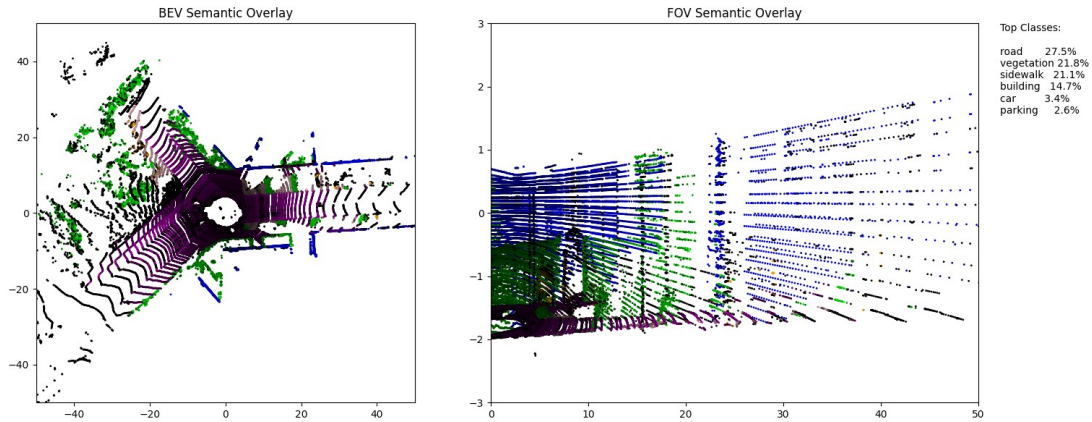
Goal

Create a **single, self-contained figure** that:

- shows spatial + forward views.
- preserves semantic meaning.
- communicates proxy behavior.
- includes interpretation context.

```python
def render_final_semantic_board(xyz, semantic_labels, proxy_norm):

    idx = deterministic_subsample_indices(len(xyz))

    xyz_sub = xyz[idx]
    proxy_sub = proxy_norm[idx]
    labels_sub = semantic_labels[idx]

    base_colors = map_semantic_to_color(labels_sub)
    final_colors = base_colors * proxy_sub[:, None]

    fig = plt.figure(figsize=(18, 6))
    gs = fig.add_gridspec(1, 3, width_ratios=[2, 2, 1])

    # BEV
    ax_bev = fig.add_subplot(gs[0, 0])
    ax_bev.scatter(xyz_sub[:, 0], xyz_sub[:, 1], c=final_colors, s=1)
    ax_bev.set_xlim(-50, 50)
    ax_bev.set_ylim(-50, 50)
    ax_bev.set_title("BEV Semantic Overlay")
    ax_bev.set_aspect('equal')

    # FOV
    ax_fov = fig.add_subplot(gs[0, 1])
    ax_fov.scatter(xyz_sub[:, 0], xyz_sub[:, 2], c=final_colors, s=1)
    ax_fov.set_xlim(0, 50)
    ax_fov.set_ylim(-3, 3)
    ax_fov.set_title("FOV Semantic Overlay")

    # INFO
    ax_info = fig.add_subplot(gs[0, 2])
    ax_info.axis('off')

    counts = Counter(labels_sub)
    total = sum(counts.values())

    lines = ["Top Classes:\n"]
    for cls, cnt in sorted(counts.items(), key=lambda x: -x[1])[:6]:
        if cls in CLASS_NAME_MAP:
            pct = 100 * cnt / total
            lines.append(f"{CLASS_NAME_MAP[cls]:10} {pct:4.1f}%")

    ax_info.text(0, 1, "\n".join(lines), va='top')

    plt.tight_layout()
    plt.show()
```

```
# TEST FINAL
render_final_semantic_board(xyz_test, semantic_test, PR_test_norm)
```



Conclusion: Visualization and Artifact Generation

This notebook completes the visualization layer of the Reflect-Aug-Seg pipeline.

What Was Built

We constructed a consistent and reproducible rendering system for:

- Bird's-Eye View (BEV) visualization.
- Forward View (FOV) visualization.
- Signal comparison boards (geometry, intensity, proxy).
- Semantic reflectivity overlays (color = class, brightness = proxy).
- Structured visualization boards with legend and class statistics.
- Frame generation pipeline for temporal sequences.
- GIF-based visual artifacts for motion analysis.

All components were built under strict constraints:

- fixed spatial limits.
- fixed normalization across frames.
- deterministic subsampling.
- no per-frame visual manipulation.

What Was Observed

Across both static frames and temporal sequences:

- pseudo-reflectivity ($\hat{\rho} = I \cdot R$) redistributes signal strength across range.
- far-range structures become more visible compared to raw intensity.
- within-class variation emerges when brightness is modulated by the proxy.
- semantic identity remains preserved under proxy modulation.
- temporal behavior remains stable without artificial smoothing.

These observations are consistent with earlier findings:

- the proxy is **meaningful but not dominant**.
- improvements are **scene-dependent**.
- behavior remains **range-influenced and heuristic**.

This visualization pipeline does not imply:

- calibrated reflectivity recovery.
- material identification.
- universal improvement over raw intensity.
- segmentation performance gains.

The proxy remains:

> a lightweight, range-aware augmentation signal.
> useful for interpretability, not physical reconstruction.

Final Takeaway

This notebook demonstrates that:

> careful visualization, when done under controlled conditions, can reveal meaningful structure in LiDAR signals without distorting the underlying data.

The pseudo-reflectivity proxy:

- does not replace raw intensity.
- but provides an additional perspective.

- that enhances interpretability across space and time.